Übungsblatt 4 – Erklärungen

VC Logik

Teil 0: Grundlagen – alles, was wir brauchen

Bevor wir uns an die Aufgaben setzen, schauen wir uns die Werkzeuge an, die wir brauchen. Einige Begriffe kennen wir schon aus den vorherigen Blättern – die frischen wir kurz auf. Andere Konzepte sind neu und werden in Ruhe eingeführt. Am Ende von Teil 0 hast du alle Bausteine, um die drei Aufgaben selbst zu lösen.

0.1 Aussagenlogische Formel – kurze Auffrischung

Eine aussagenlogische Formel ist ein Ausdruck, der aus **Variablen** und **Operatoren** aufgebaut ist. Jede Variable steht für eine ja/nein-Frage und kann den Wert 0 (falsch) oder 1 (wahr) annehmen.

Symbol	**Name**	**Bedeutung**
$\neg A$	Negation	„Nicht A“
$A \wedge B$	Konjunktion	„A und B“ (beide wahr)
$A \vee B$	Disjunktion	„A oder B“ (mind. eines wahr)
$A \rightarrow B$	Implikation	„Wenn A, dann B“
$A \leftrightarrow B$	Biimplikation	„Genau dann A, wenn B“

Als Variablennamen schreibt man oft $p_1, p_2, p_3, \ldots$ (wenn man viele braucht) oder einfach $A, B, C, \ldots$ (wenn man nur wenige braucht).

Merke

Ein **Modell** einer Formel Φ ist eine Belegung der Variablen mit 0/1, die die Formel wahr macht. Eine Formel kann 0, 1 oder *sehr viele* Modelle haben.

Analogie

Eine Formel ist wie eine Türschließanlage mit vielen Schaltern. Jedes „Modell“ ist eine Einstellung der Schalter, bei der die Tür aufgeht. Manche Türen haben nur einen einzigen richtigen Code (ein Modell). Andere lassen sich mit vielen verschiedenen Codes öffnen (viele Modelle). Wieder andere können gar nicht geöffnet werden (kein Modell – die Formel ist unerfüllbar).

0.2 Warum Logik für „richtige“ Probleme?

Jetzt kommt der Punkt, der Logik richtig mächtig macht:

Die große Idee

Viele Probleme aus der echten Welt lassen sich als Formel *codieren*. Die **Modelle der Formel entsprechen dann genau den Lösungen des Problems**. Ein Programm, das Modelle findet (ein *SAT-Solver*), löst also das Problem.

Analogie

Die Aussagenlogik ist eine Art **universelle Puzzle-Sprache**. Egal ob Sudoku, Stundenplan, Schaltkreis-Verifikation oder N-Damen-Problem – alles kann man in dieselbe Sprache übersetzen. Und die Computer, die diese Sprache verstehen, sind heutzutage unglaublich schnell (SAT-Solver lösen Formeln mit Millionen von Variablen).

In diesem Blatt werden wir drei Beispiele für dieses Prinzip sehen: Sudoku (Aufgabe 1), ein Schaltkreis (Aufgabe 2) und das Bishop-Problem auf dem Schachbrett (Aufgabe 3).

0.3 Encoding-Muster für Kombinatorik

Wie codiert man ein Problem? Es gibt ein wiederkehrendes Rezept:

Strategie: Problem → Formel

1. **Eine Bool'sche Variable pro Entscheidung.** Beispiele: „Steht Ziffer n in Feld (i, j)?“, „Steht ein Läufer auf Feld (i, j)?“, „Hat Draht o_1 den Wert 1?“.
2. **Jede Regel des Problems als Teilformel.** „Mindestens einer muss“ wird zu einer $\vee$-Verknüpfung. „Höchstens einer darf“ wird zu einer UND-Verknüpfung von Ausschluss-Paaren.
3. **Alle Teilformeln mit $\wedge$ verbinden.** Die Gesamtformel ist die Konjunktion aller Regeln – alle Regeln müssen gleichzeitig gelten.

Analogie

Wir übersetzen das Problem wie einen **Anforderungskatalog**. Ein Bauherr hätte ihn so geschrieben:

- „In jedem Raum muss mindestens eine Lampe sein.“
- „In jedem Raum darf höchstens eine Hauptlampe sein.“
- „Jeder Schalter gehört zu genau einer Lampe.“

Das ist schon fast Logik: „mindestens eine“, „höchstens eine“, „genau eine“. Diese drei Muster kehren immer wieder.

0.4 At-most-one (AMO) – das wichtigste Werkzeug

Dieses Muster brauchen wir in allen drei Aufgaben. Es ist so wichtig, dass es einen eigenen Namen hat.

Merke

At-most-one (AMO). Von k Variablen $x_1, x_2, \ldots, x_k$ soll *höchstens eine* wahr sein. Als Formel mit paarweiser Enkodierung:

$$\mathrm{AMO}(x_1, \ldots, x_k) \;\equiv\; \bigwedge_{1 \le i < j \le k} \neg(x_i \wedge x_j).$$

Anzahl Klauseln: $\binom{k}{2} = \frac{k(k-1)}{2}$.

Warum funktioniert das? Die AMO-Eigenschaft „höchstens eine wahr“ ist gleichbedeutend mit: „es gibt kein Paar, in dem beide wahr sind“. Die Teilformel $\neg(x_i \wedge x_j)$ verbietet genau dieses Paar. Wenn wir *alle* Paare verbieten, kann nirgends ein Paar auftauchen – also sind entweder 0 oder 1 Variablen wahr.

Beispiel mit $k = 3$. Bei drei Variablen x_1, x_2, x_3 brauchen wir $\binom{3}{2} = 3$ Klauseln:

$$\neg(x_1 \wedge x_2) \;\wedge\; \neg(x_1 \wedge x_3) \;\wedge\; \neg(x_2 \wedge x_3).$$

Probier es aus: Wenn $x_1 = x_2 = 1$, dann ist die erste Klausel falsch, also die ganze Formel falsch. Gut, verboten. Wenn nur $x_1 = 1$ ist (und die anderen 0), dann ist jede Klausel der Form $\neg(1 \wedge 0) = \neg 0 = 1$, also wahr. Passt, erlaubt.

Analogie

Aus dem Alltag: „Von den drei Kuchenstücken darf höchstens eines gegessen werden.“ Das heißt: nicht Kuchen-A *und* Kuchen-B zugleich, nicht Kuchen-A *und* Kuchen-C zugleich, nicht Kuchen-B *und* Kuchen-C zugleich. Drei Ausschlüsse, fertig. Das ist die AMO-Enkodierung.

0.5 At-least-one und Exactly-one

Zwei Geschwister von AMO:

Merke

At-least-one (ALO). „Mindestens eine ist wahr“:

$$\mathrm{ALO}(x_1, \ldots, x_k) \;\equiv\; x_1 \vee x_2 \vee \ldots \vee x_k.$$

Das ist eine einzige große Klausel.

Exactly-one (EO). „Genau eine ist wahr“:

$$\mathrm{EO}(x_1, \ldots, x_k) \;\equiv\; \mathrm{ALO}(\ldots) \;\wedge\; \mathrm{AMO}(\ldots).$$

Also „mindestens eine“ *und* „höchstens eine“ zusammen.

Analogie

ALO ist: „Mindestens ein Fenster im Haus ist offen.“ Es reicht schon, wenn *irgendeins* offen ist.

EO ist: „Es ist genau eine Tasche am Haken.“ Nicht null, nicht zwei, genau eine. Also braucht man zwei Einzelregeln gleichzeitig: mindestens eine (ALO) und höchstens eine (AMO).

0.6 Schaltkreise als Formel

Für Aufgabe 2: Ein Schaltkreis besteht aus **Gates** (Bauteilen), die durch Drähte verbunden sind. Jedes Gate hat Eingänge und einen Ausgang. Der Ausgang hängt durch eine fest vorgegebene Regel von den Eingängen ab.

Strategie: Schaltkreis → Formel

Für *jeden* Draht (insbesondere den Ausgang jedes Gates) schreibt man eine Äquivalenz

$$\text{(Draht-Variable)} \leftrightarrow \text{(Funktion der Eingangs-Variablen)}.$$

Die Gesamtformel ist die Konjunktion all dieser Äquivalenzen.

Analogie

Jedes Gate ist wie ein kleiner Mitarbeiter, der eine feste Regel befolgt. Ein UND-Gate sagt: „Wenn beide Inputs 1 sind, gebe ich 1 aus, sonst 0.“ Um den ganzen Betrieb zu verstehen, sammeln wir die Regel jedes einzelnen Mitarbeiters (jede Gate-Regel) und kleben sie alle mit UND zusammen. Nur wenn *jeder* Mitarbeiter seine Regel einhält, ist der Betrieb korrekt.

Wichtig: Es gibt sichtbare Variablen (Eingang/Ausgang des ganzen Schaltkreises) und **Hilfsvariablen** für interne Drähte. Die Hilfsvariablen bekommen zusätzliche Namen wie $o_1, o_2, \ldots$, damit man jede einzelne Gate-Regel sauber hinschreiben kann.

0.7 Erfüllbarkeitstester – der Fragen-Sklave

Aus dem letzten Blatt: Ein **Erfüllbarkeitstester** (SAT-Solver) ist ein Programm, das zu einer gegebenen Formel Φ entscheidet, ob sie ein Modell hat (SAT, erfüllbar) oder nicht (UNSAT, unerfüllbar).

Zwei wichtige Fragen-Muster

- **„Kann X gelten?“** $\Rightarrow$ Prüfe $\Phi \wedge X$ auf SAT. Erfüllbar $\Rightarrow$ Ja, X ist möglich. Unerfüllbar $\Rightarrow$ Nein, unter Φ ist X ausgeschlossen.
- **„Gilt immer Y?“** $\Rightarrow$ Prüfe $\Phi \wedge \neg Y$ auf SAT. Unerfüllbar $\Rightarrow$ Ja, Y gilt immer. Erfüllbar $\Rightarrow$ Nein, Gegenbeispiel gefunden.

Analogie

Ein Erfüllbarkeitstester ist wie ein treuer Diener, der nur eine einzige Frage versteht: „Gibt es für diese Formel eine Belegung, die sie wahr macht?“ Alle anderen Fragen über Logik-Formeln lassen sich auf diese eine Frage reduzieren – man muss die Frage nur geschickt umformulieren, wie oben beschrieben.

Warum funktioniert „Gilt immer Y?“ über $\neg Y$? „Y gilt immer (unter Φ)“ bedeutet: In jedem Modell von Φ ist Y wahr. Das Gegenteil wäre: Es gibt ein Modell von Φ, in dem Y falsch ist – also ein Modell von $\Phi \wedge \neg Y$. Findet der Tester *kein* solches Modell (UNSAT), dann gilt Y tatsächlich immer.

0.8 Schach und Diagonalen

Für Aufgabe 3: Ein Schachbrett hat n Zeilen und n Spalten. Die Felder bezeichnen wir mit (i, j), wobei i die Zeilennummer und j die Spaltennummer ist, beide von 1 bis n.

Schachfiguren – Bewegungsmuster

- **Rook (Turm):** bewegt sich entlang Zeilen und Spalten (horizontal/vertikal).
- **Bishop (Läufer):** bewegt sich entlang Diagonalen (schräg).
- **Queen (Dame):** bewegt sich wie Rook UND Bishop kombiniert.

Für den Läufer (Bishop) brauchen wir eine klare Definition von „Diagonale“:

Diagonalen

- **Hauptdiagonale:** Alle Felder (i, j) mit demselben Wert $i - j$. Die führt von links oben nach rechts unten. Beispiel: $(1, 1), (2, 2), (3, 3), (4, 4)$ haben alle $i - j = 0$.
- **Gegendiagonale:** Alle Felder (i, j) mit demselben Wert $i + j$. Die führt von rechts oben nach links unten. Beispiel: $(1, 4), (2, 3), (3, 2), (4, 1)$ haben alle $i + j = 5$.

Auf einem $n \times n$-Brett gibt es jeweils $2n - 1$ Hauptdiagonalen und $2n - 1$ Gegendiagonalen.

Analogie

Diagonalen sind wie **Straßen, die quer über das Brett führen**. Jedes Feld liegt an einer Kreuzung von genau zwei Straßen: einer Hauptdiagonale und einer Gegendiagonale. Der Läufer „bewohnt“ also zwei Straßen gleichzeitig. Zwei Läufer können sich nur dann schlagen, wenn sie auf derselben Straße stehen – also dieselbe Hauptdiagonale (gleiches $i - j$) oder dieselbe Gegendiagonale (gleiches $i + j$) teilen.

Klein-Beispiel $n = 4$**.** Die Werte von $i - j$ liegen zwischen $-(n - 1) = -3$ und $n-1 = 3$, also $i-j \in \{-3, -2, -1, 0, 1, 2, 3\}$ – das sind $7 = 2n-1$ Hauptdiagonalen. Analog $i + j \in \{2, 3, 4, 5, 6, 7, 8\}$ – auch 7 Gegendiagonalen. Schauen wir uns die Hauptdiagonale $i - j = 0$ an: Das sind die Felder $(1, 1), (2, 2), (3, 3), (4, 4)$, die vom linken oberen Eck bis zum rechten unteren Eck verlaufen.

1 Aufgabe 1: Sudoku als Formel

Aufgabe in einfachen Worten

Wir haben ein 4×4-Sudoku-Feld. Jede Zelle soll eine Ziffer von 1 bis 4 enthalten. Zusatzregeln: In jeder Zeile, in jeder Spalte und in jeder der vier 2×2-Regionen darf jede Ziffer *höchstens einmal* vorkommen. Gesucht: eine aussagenlogische Formel Φ, deren Modelle genau den gültigen Sudoku-Lösungen entsprechen.

Analogie

Denk an Sudoku wie an ein **kleines Rätsel**, bei dem du mit vier Stempeln (mit den Ziffern 1, 2, 3, 4) auf 16 Felder stempeln sollst. Die Regeln: In jeder Zeile, Spalte und Region darfst du jeden Stempel höchstens einmal benutzen. Unsere Aufgabe ist es, diese Regeln so in Logik zu gießen, dass jede gültige Stempel-Anordnung einer wahren Belegung unserer Formel entspricht.

Schritt 1: Die Variablen einführen

Pro Feld gibt es 4 Mögliche Ziffern. Und pro Ziffer gibt es 16 Felder, auf denen sie stehen könnte. Wir brauchen eine Bool'sche Variable für jede *Kombination*:

Die Variablen

Für $i, j, n \in \{1, 2, 3, 4\}$ definieren wir

$$x_{i,j,n} \;:=\; \text{“Feld } (i, j) \text{ enthält die Ziffer } n\text{”}.$$

Das ergibt $4 \cdot 4 \cdot 4 = 64$ Variablen.

Analogie

Genauso wie in Blatt 2 die Lichtschalter-Analogie: Jede Variable ist ein kleiner Schalter, der „an“ (1) oder „aus“ (0) steht. Der Schalter $x_{1,1,3}$ steht auf „an“, wenn im Feld in der oberen linken Ecke die Ziffer 3 geschrieben wird. Steht der Schalter auf „aus“, dann steht dort irgendetwas anderes.

Kleines Beispiel. Wenn das Sudoku so aussieht:

1	2	3	4
3	4	1	2
2	1	4	3
4	3	2	1

Dann ist $x_{1,1,1} = 1$ (im Feld oben links steht die 1), $x_{1,1,2} = 0, x_{1,1,3} = 0, x_{1,1,4} = 0$ (dort steht keine 2, 3 oder 4). Analog $x_{1,2,2} = 1$ (im zweiten Feld der ersten Zeile steht die 2).

Schritt 2: Feld-Layout und Regionen

Das 4×4-Brett wird in vier 2×2-Regionen geteilt (wie bei „großem“ 9×9-Sudoku mit 3×3-Regionen).

$(1,1)$	$(1,2)$	$(1,3)$	$(1,4)$
$(2,1)$	$(2,2)$	$(2,3)$	$(2,4)$
$(3,1)$	$(3,2)$	$(3,3)$	$(3,4)$
$(4,1)$	$(4,2)$	$(4,3)$	$(4,4)$

Die vier Regionen:

- $R_1 = \{(1,1),(1,2),(2,1),(2,2)\}$ (links oben)
- $R_2 = \{(1,3),(1,4),(2,3),(2,4)\}$ (rechts oben)
- $R_3 = \{(3,1),(3,2),(4,1),(4,2)\}$ (links unten)
- $R_4 = \{(3,3),(3,4),(4,3),(4,4)\}$ (rechts unten)

Schritt 3: Constraints schrittweise aufbauen

Jetzt übersetzen wir die Sudoku-Regeln nacheinander in Logik. Wir stellen uns vor, dass wir einen Anforderungskatalog abarbeiten – eine Regel nach der anderen.

Analogie

Wie beim **Kuchenbacken nach Rezept**: Erst Mehl, dann Zucker, dann Eier. Jede einzelne Zutat ist eine kleine Formel. Am Ende rühren wir alles zusammen (mit $\wedge$) und erhalten den fertigen Teig (die Gesamtformel).

(C1) Jedes Feld enthält mindestens eine Ziffer

Das ist ein **At-least-one**-Constraint: Die Variablen für ein und dasselbe Feld $(x_{i,j,1}, x_{i,j,2}, x_{i,j,3}, x_{i,j,4})$ sollen nicht alle 0 sein.

$$C_1 \equiv \bigwedge_{i=1}^{4} \bigwedge_{j=1}^{4} \big(x_{i,j,1} \vee x_{i,j,2} \vee x_{i,j,3} \vee x_{i,j,4}\big).$$

Für jedes der 16 Felder eine Disjunktion mit 4 Literalen. Insgesamt also 16 solche Disjunktionen, die alle mit $\wedge$ verknüpft sind.

Konkretes Beispiel für Feld $(1,1)$**:**

$$x_{1,1,1} \vee x_{1,1,2} \vee x_{1,1,3} \vee x_{1,1,4}.$$

Das heißt: „Im Feld $(1,1)$ steht mindestens eine der Ziffern 1, 2, 3, 4.“ Für jedes andere Feld analog.

(C2) Jedes Feld enthält höchstens eine Ziffer

Hier kommt **AMO** zum Einsatz: Für ein Feld (i,j) soll nicht gleichzeitig „hier steht Ziffer n“ und „hier steht Ziffer m“ gelten, wenn $n \neq m$.

$$C_2 \equiv \bigwedge_{i=1}^{4} \bigwedge_{j=1}^{4} \bigwedge_{1 \le n < m \le 4} \neg\big(x_{i,j,n} \wedge x_{i,j,m}\big).$$

Pro Feld gibt es $\binom{4}{2} = 6$ Paare (n,m), also 6 Klauseln. Insgesamt $16 \cdot 6 = 96$ Klauseln.

Konkretes Beispiel für Feld $(1,1)$**:** Die 6 Paare sind $(1,2), (1,3), (1,4), (2,3), (2,4), (3,4)$.

$$\begin{aligned}
&\neg(x_{1,1,1} \wedge x_{1,1,2}) \;\wedge\; \neg(x_{1,1,1} \wedge x_{1,1,3}) \\
&\wedge\; \neg(x_{1,1,1} \wedge x_{1,1,4}) \;\wedge\; \neg(x_{1,1,2} \wedge x_{1,1,3}) \\
&\wedge\; \neg(x_{1,1,2} \wedge x_{1,1,4}) \;\wedge\; \neg(x_{1,1,3} \wedge x_{1,1,4}).
\end{aligned}$$

Merke

(C1) und (C2) zusammen erzwingen: Jedes Feld enthält **genau** eine Ziffer. Das ist der Exactly-one-Constraint – aber wir haben ihn durch Kombination von ALO und AMO realisiert.

(C3) Jede Ziffer höchstens einmal pro Zeile

Jetzt wechselt die Perspektive: Wir fixieren eine Zeile i und eine Ziffer n und schauen auf die 4 Variablen $x_{i,1,n}, x_{i,2,n}, x_{i,3,n}, x_{i,4,n}$. Davon darf höchstens eine 1 sein – wieder AMO.

$$C_3 \;\equiv\; \bigwedge_{i=1}^{4} \bigwedge_{n=1}^{4} \bigwedge_{1 \le j < k \le 4} \neg\big(x_{i,j,n} \wedge x_{i,k,n}\big).$$

Pro (Zeile, Ziffer)-Kombi gibt es $\binom{4}{2} = 6$ Paare. Insgesamt $4 \cdot 4 \cdot 6 = 96$ Klauseln.
Konkretes Beispiel für Zeile 1 **und Ziffer** 3**:**

$$\begin{aligned}
&\neg(x_{1,1,3} \wedge x_{1,2,3}) \;\wedge\; \neg(x_{1,1,3} \wedge x_{1,3,3}) \\
&\wedge\; \neg(x_{1,1,3} \wedge x_{1,4,3}) \;\wedge\; \neg(x_{1,2,3} \wedge x_{1,3,3}) \\
&\wedge\; \neg(x_{1,2,3} \wedge x_{1,4,3}) \;\wedge\; \neg(x_{1,3,3} \wedge x_{1,4,3}).
\end{aligned}$$

Diese Klauseln sagen: „In Zeile 1 gibt es nicht zwei Felder mit der Ziffer 3.“

(C4) Jede Ziffer höchstens einmal pro Spalte

Analog zu (C3), aber mit fixer Spalte j und variierender Zeile.

$$C_4 \;\equiv\; \bigwedge_{j=1}^{4} \bigwedge_{n=1}^{4} \bigwedge_{1 \le i < k \le 4} \neg\big(x_{i,j,n} \wedge x_{k,j,n}\big).$$

Auch hier $4 \cdot 4 \cdot 6 = 96$ Klauseln.

(C5) Jede Ziffer höchstens einmal pro Region

Und zum Schluss das gleiche Muster für die vier Regionen:

$$C_5 \;\equiv\; \bigwedge_{R \in \{R_1, R_2, R_3, R_4\}} \bigwedge_{n=1}^{4} \bigwedge_{\substack{(i,j),(k,l) \in R \\ (i,j) < (k,l)}} \neg\big(x_{i,j,n} \wedge x_{k,l,n}\big).$$

(Das Paar $(i,j) < (k,l)$ meint: wir nehmen jedes ungeordnete Paar nur einmal, z. B. in lexikografischer Ordnung.) Pro (Region, Ziffer)-Kombi gibt es $\binom{4}{2} = 6$ Paare, insgesamt wieder $4 \cdot 4 \cdot 6 = 96$.

Konkretes Beispiel für Region $R_1 = \{(1,1),(1,2),(2,1),(2,2)\}$ und Ziffer 2:

$$\begin{aligned}
&\neg(x_{1,1,2} \wedge x_{1,2,2}) \;\wedge\; \neg(x_{1,1,2} \wedge x_{2,1,2}) \\
&\wedge\; \neg(x_{1,1,2} \wedge x_{2,2,2}) \;\wedge\; \neg(x_{1,2,2} \wedge x_{2,1,2}) \\
&\wedge\; \neg(x_{1,2,2} \wedge x_{2,2,2}) \;\wedge\; \neg(x_{2,1,2} \wedge x_{2,2,2}).
\end{aligned}$$

Schritt 4: Die Gesamtformel

Alles zusammen:

$$\boxed{\Phi \;\equiv\; C_1 \wedge C_2 \wedge C_3 \wedge C_4 \wedge C_5}$$

Constraint	**Bedeutung**	**Klauseln**
C_1	Mindestens eine Ziffer pro Feld	16
C_2	Höchstens eine Ziffer pro Feld (AMO)	96
C_3	Keine Ziffer zweimal pro Zeile (AMO)	96
C_4	Keine Ziffer zweimal pro Spalte (AMO)	96
C_5	Keine Ziffer zweimal pro Region (AMO)	96
Gesamt		**400**

Die Formel Φ hat also 400 Klauseln über 64 Variablen. Jedes Modell dieser Formel entspricht einer gültigen Sudoku-Lösung.

Warum „höchstens“ statt „genau“?

Die Aufgabenstellung sagt bewusst nur „höchstens“, nicht „genau“. Dadurch erlaubt die Formel auch unvollständig gefüllte Bretter – zum Beispiel das leere Brett (keine Ziffern überall) ist kein Modell, weil (C1) fordert, dass jedes Feld gefüllt ist. Aber wenn man (C1) wegließe, wäre auch das leere Brett ein Modell. Mit (C1)+(C2) ist jedes Feld trotzdem mit genau einer Ziffer belegt, und die Einschränkungen (C3)-(C5) sorgen dafür, dass keine Zeile, Spalte oder Region doppelte Ziffern hat.

Goldene Regel des Encodings

Dieses Muster funktioniert für fast jedes kombinatorische Problem:

- N-Damen-Problem: n^2 Variablen für „Dame auf Feld (i,j)“, AMO pro Zeile/-Spalte/Diagonale.
- Graph-Färbung: Variable $c_{v,n}$ für „Knoten v hat Farbe n“, EO pro Knoten, Ausschluss bei Nachbarknoten.
- Aufzugs-Scheduling: Variable pro (Person, Stockwerk, Zeit), EO pro Person zu jedem Zeitpunkt, Kapazitäts-Constraints pro Aufzug.

Immer das gleiche Rezept: **Variablen pro Entscheidung + Constraints pro Gruppe**.

2 Aufgabe 2: Schaltkreis als Formel

Aufgabe in einfachen Worten

Wir haben einen Schaltkreis mit drei Eingängen A, B, C und drei Ausgängen D, E, F. Dazwischen gibt es „Gates“ (Bauteile), die aus ihren Eingangssignalen einen Ausgang berechnen. Der Clou bei dieser Aufgabe: Die Gates rechnen nicht mit klassischen UND/ODER, sondern mit *Summen-Bedingungen* – zum Beispiel „Output = 1 genau dann, wenn die Summe der Inputs größer als 1 ist“.

Gesucht: eine aussagenlogische Formel, deren Modelle genau den zulässigen Belegungen (A, B, C, D, E, F) entsprechen. Außerdem sollen wir drei Fragen zum Schaltkreis beantworten, indem wir die Formel einem Erfüllbarkeitstester vorlegen.

Schritt 1: Gate-Semantik verstehen

Die Notation $[> k]$ oder $[= k]$ auf einem Gate bedeutet: Der Output ist 1 genau dann, wenn die **Summe der Inputs** (als Zahlen gelesen, also binär 0/1) diese Bedingung erfüllt.

Summen-Bedingungen als Bool'sche Ausdrücke

Für binäre Inputs (je 0 oder 1) gilt:

- $[> 1](A, B)$: $A + B > 1 \Leftrightarrow$ beide sind 1 $\Leftrightarrow A \wedge B$.
- $[= 1](A, C)$: $A + C = 1 \Leftrightarrow$ genau einer ist 1 $\Leftrightarrow (A \wedge \neg C) \vee (\neg A \wedge C)$. Das ist *XOR*.
- $[> 0](B, C)$: $B + C > 0 \Leftrightarrow$ mindestens einer ist 1 $\Leftrightarrow B \vee C$.
- $[= 0](x)$: $x = 0 \Leftrightarrow \neg x$.
- $[> 2](o_1, o_2, o_3)$: Summe von drei binären Werten größer als 2 $\Leftrightarrow$ alle drei sind 1 $\Leftrightarrow o_1 \wedge o_2 \wedge o_3$.
- $[> 1](o_2, o_3)$: $o_2 + o_3 > 1 \Leftrightarrow$ beide sind 1 $\Leftrightarrow o_2 \wedge o_3$.

Analogie

Ein Summen-Gate ist wie eine **Abstimmung**. $[> 1]$ bedeutet „mehr als ein Ja-Stimmer“, $[= 0]$ bedeutet „kein Ja-Stimmer“, $[> 2]$ bei drei Stimmern bedeutet „alle stimmen ja“ usw. Der Output ist 1, wenn das Abstimmungsergebnis die Vorgabe erfüllt.

Warum ist bei zwei binären Inputs $[> 1]$ dasselbe wie $\wedge$? Die möglichen Summen sind 0, 1 oder 2. „> 1“ heißt also $= 2$, also beide $= 1$. Das ist die UND-Verknüpfung.

Warum ist $[= 1]$ bei zwei Inputs XOR? Die Summe ist 0 (beide 0), 1 (genau einer 1) oder 2 (beide 1). „$= 1$“ trifft nur den mittleren Fall zu. Das ist per Definition das Exclusive-Or.

Schritt 2: Verkabelung ablesen

Schauen wir uns den Schaltkreis genau an. Der Input A geht in das Gate oben links, zugleich in das mittlere Gate. B geht in das Gate oben links und in das untere Gate. C geht in das mittlere und in das untere Gate.

Die drei linken Gates produzieren Hilfsoutputs, die wir o_1, o_2, o_3 nennen. Diese Outputs gabeln sich und gehen zum Gate „oben rechts“ (berechnet D), zum Gate „in der Mitte“ (berechnet E) und zum Gate „unten rechts“ (berechnet F).

Gate	**Inputs**	**Bedingung**	**Output**
$[> 1]$ (oben links)	A, B	$A + B > 1$	o_1
$[= 1]$ (mitte links)	A, C	$A + C = 1$	o_2
$[> 0]$ (unten links)	B, C	$B + C > 0$	o_3
$[= 0]$ (oben rechts)	o_1	$o_1 = 0$	D
$[> 2]$ (mitte rechts)	o_1, o_2, o_3	Summe > 2	E
$[> 1]$ (unten rechts)	o_2, o_3	$o_2 + o_3 > 1$	F

Merke

Die drei Hilfsvariablen o_1, o_2, o_3 sind interne Drähte – sie treten an den „Außenklemmen“ des Schaltkreises nicht auf, aber wir brauchen sie, um die Formel sauber aufzuschreiben. Die „sichtbaren“ Variablen sind A, B, C, D, E, F.

Schritt 3: Die Formel zusammenbauen

Für jeden Draht eine Äquivalenz. Das ergibt sechs Teilformeln, die alle mit $\wedge$ verknüpft werden:

$$\begin{aligned}
\Phi \equiv\ & \big(o_1 \leftrightarrow (A \wedge B)\big) \\
\wedge\ & \big(o_2 \leftrightarrow ((A \wedge \neg C) \vee (\neg A \wedge C))\big) \\
\wedge\ & \big(o_3 \leftrightarrow (B \vee C)\big) \\
\wedge\ & \big(D \leftrightarrow \neg o_1\big) \\
\wedge\ & \big(E \leftrightarrow (o_1 \wedge o_2 \wedge o_3)\big) \\
\wedge\ & \big(F \leftrightarrow (o_2 \wedge o_3)\big).
\end{aligned}$$

Analogie

Jede Zeile der Formel ist wie ein **Vertragsparagraph**. „Der Draht o_1 führt genau dann Strom, wenn sowohl A als auch B Strom führen.“ Jeder Paragraph muss gleichzeitig gelten (das $\wedge$ zwischen den Zeilen). Nur wenn alle Paragraphen erfüllt sind, ist die Gesamtkonfiguration des Schaltkreises zulässig.

Schritt 4: Die drei Fragen beantworten

Alle drei Fragen sind Fragen der Art „Kann … gelten?“ oder „Gilt immer … ?“. Wir lösen sie mit dem Erfüllbarkeitstester (siehe Teil 0.7).

	Frage	Dem Tester geben
(a)	Kann $F = 1$ sein?	$\Phi \wedge F$ auf SAT testen
(b)	Kann $E = A$ sein?	$\Phi \wedge (E \leftrightarrow A)$ auf SAT testen
(c)	Gilt $F = 1 \Leftrightarrow D = 0$?	$\Phi \wedge \neg(F \leftrightarrow \neg D)$ auf UNSAT testen

Statt einen Solver laufen zu lassen, können wir es auch direkt mit konkreten Belegungen nachprüfen – das ist übersichtlicher:

(a) Kann $F = 1$ sein?

Methode. Wir suchen eine Belegung (A, B, C), für die sich $F = 1$ ergibt.

Versuch: $A = 1, B = 1, C = 0$**.** Dann:

- $o_1 = A \wedge B = 1 \wedge 1 = 1$.
- $o_2 = (A \wedge \neg C) \vee (\neg A \wedge C) = (1 \wedge 1) \vee (0 \wedge 0) = 1 \vee 0 = 1$.
- $o_3 = B \vee C = 1 \vee 0 = 1$.
- $F = o_2 \wedge o_3 = 1 \wedge 1 = 1$. ✓

Antwort: $\boxed{\text{Ja, } F = 1 \text{ ist möglich.}}$

(b) Kann $E = A$ sein?

Methode. Wir suchen eine Belegung, in der E und A denselben Wert haben.

Trivial-Versuch: $A = 0, B = 0, C = 0$**.** Dann:

- $o_1 = 0 \wedge 0 = 0$, $o_2 = (0 \wedge 1) \vee (1 \wedge 0) = 0$, $o_3 = 0 \vee 0 = 0$.
- $E = o_1 \wedge o_2 \wedge o_3 = 0 = A$. ✓

Das ist aber ein billiger Sieg. Spannender: gibt es auch eine Belegung mit $A = 1$ und $E = 1$?

Versuch: $A = 1, B = 1, C = 0$**.** (Dieselbe wie bei (a).)

- $o_1 = 1, o_2 = 1, o_3 = 1$.
- $E = o_1 \wedge o_2 \wedge o_3 = 1 = A$. ✓

Antwort: $\boxed{\text{Ja, } E = A \text{ ist möglich – sogar auf zwei verschiedene Arten.}}$

(c) Gilt $F = 1 \Leftrightarrow D = 0$?

Das ist eine *immer*-Frage. Wir fragen: Stimmt es, dass in *jedem* zulässigen Zustand F und $\neg D$ übereinstimmen?

Methode. Wenn die Äquivalenz immer gilt, dann ist ihr Gegenteil nie erfüllbar, also $\Phi \wedge \neg(F \leftrightarrow \neg D)$ unerfüllbar. Findet man ein **Gegenbeispiel**, dann gilt die Äquivalenz nicht.

Suche nach Gegenbeispiel: $A = 1, B = 1, C = 1$**.**

- $o_1 = A \wedge B = 1 \wedge 1 = 1$. Also $D = \neg o_1 = 0$.
- $o_2 = (A \wedge \neg C) \vee (\neg A \wedge C) = (1 \wedge 0) \vee (0 \wedge 1) = 0$.
- $o_3 = B \vee C = 1 \vee 1 = 1$.
- $F = o_2 \wedge o_3 = 0 \wedge 1 = 0$.

In diesem Zustand gilt $D = 0$, aber $F = 0$. Die rechte Seite $D = 0$ ist wahr, die linke Seite $F = 1$ ist falsch. Damit ist die Biimplikation $F = 1 \Leftrightarrow D = 0$ verletzt.

Antwort: $\boxed{\text{Nein, die Äquivalenz gilt NICHT generell.}}$ Gegenbeispiel: $(A, B, C) = (1, 1, 1)$, hier ist $D = 0$, aber $F = 0$.

Intuition zum Gegenbeispiel

$D = 0$ heißt, dass oben links beide Inputs 1 sind ($A = B = 1$). $F = 1$ dagegen braucht, dass beide Outputs o_2 und o_3 gleich 1 sind – und o_2 ist ein XOR-Gate über A und C, das wird 0, sobald $A = C$. Also: sobald $A = B = C$, widerspricht sich $D = 0$ und $F = 1$. Ein klares Gegenbeispiel.

3 Aufgabe 3: Bishop Independence Problem

Aufgabe in einfachen Worten

Wir haben ein $n \times n$-Schachbrett. Gesucht sind alle Konfigurationen, in denen Läufer (Bishops) so platziert sind, dass *keine zwei einander schlagen können*. Zur Erinnerung: Der Läufer zieht diagonal. Zwei Läufer schlagen sich also genau dann, wenn sie auf derselben Diagonale stehen.

Der Auftrag: Schreibe ein Skript, das für gegebenes n eine aussagenlogische Formel produziert. Die Modelle der Formel sollen genau die gültigen Konfigurationen sein.

Schritt 1: Vergleich mit Rook und Queen

In der Vorlesung wurden verwandte Probleme besprochen:

Figur	**Schlägt auf**
Rook (Turm)	Zeile, Spalte
Bishop (Läufer)	Diagonalen (haupt + gegen)
Queen (Dame)	Zeile, Spalte, Diagonalen

Die Encoding-Strategie ist bei allen drei gleich. Was sich ändert, sind die **Gruppen**, in denen der AMO-Constraint gilt:

- Rook: AMO pro Zeile + AMO pro Spalte.
- Bishop: AMO pro Hauptdiagonale + AMO pro Gegendiagonale.
- Queen: AMO pro Zeile + Spalte + beide Diagonalen-Arten.

Analogie

Der Läufer ist ein **„Diagonal-Turm“**: Statt sich rechts, links, oben oder unten zu bewegen, wandert er schräg. Alles, was wir für den Turm wussten (AMO pro Zeile/Spalte), übertragen wir jetzt auf die Diagonalen.

Schritt 2: Variablen und Diagonalen

Die Variablen

Für jedes Feld (i, j) mit $1 \leq i, j \leq n$ definiere

$$b_{i,j} \;:=\; \text{“Auf Feld } (i,j) \text{ steht ein Läufer”}.$$

Zur Erinnerung aus Teil 0.8: Jedes Feld (i, j) liegt auf zwei Diagonalen:

- der Hauptdiagonale mit Index $i - j$,
- der Gegendiagonale mit Index $i + j$.

Beispielbrett $n = 4$ mit den Hauptdiagonalen eingezeichnet (Zahl = Index $i - j$):

0	-1	-2	-3
1	0	-1	-2
2	1	0	-1
3	2	1	0

Felder mit dem gleichen Wert liegen auf derselben Hauptdiagonale – z. B. die vier Felder mit 0 bilden die Hauptdiagonale $(1,1),(2,2),(3,3),(4,4)$.

Dasselbe Brett mit Gegendiagonalen (Zahl = Index $i+j$):

2	3	4	5
3	4	5	6
4	5	6	7
5	6	7	8

Hier bilden z. B. die Felder mit 5 die Gegendiagonale $(1,4),(2,3),(3,2),(4,1)$.

Schritt 3: Die Constraints formulieren

Bishop-Constraint

Für jede Diagonale D (egal ob Haupt oder Gegen) mit mindestens zwei Feldern gilt: **höchstens ein Läufer auf** D. In paarweiser Enkodierung (AMO):

$$\bigwedge_{\substack{(i_1,j_1),(i_2,j_2)\in D\\(i_1,j_1)\neq(i_2,j_2)}} \neg\big(b_{i_1,j_1} \wedge b_{i_2,j_2}\big).$$

Die Gesamtformel ist die Konjunktion aller dieser Klauseln, über alle Diagonalen hinweg.

Warum keine Bedingung „mindestens ein Läufer“? Die Aufgabe fragt nur nach *Unabhängigkeit*, nicht nach einer Mindestanzahl. Das leere Brett ist eine triviale Lösung (keine zwei Läufer schlagen sich, wenn es gar keine gibt). Wenn wir zusätzlich verlangen wollten, dass mindestens k Läufer stehen, müssten wir dafür extra Constraints hinzufügen.

Schritt 4: Das Python-Skript

Das Skript geht in zwei Phasen vor: erst alle Diagonalen sammeln, dann pro Diagonale die AMO-Klauseln erzeugen.

```python
def bishop_independence(n):
    """
    Erzeugt die aussagenlogische Formel (als Liste von Klauseln) fuer
    das Bishop Independence Problem auf einem n x n Schachbrett.

    Variablen: b_{i,j} (1 <= i,j <= n) steht fuer
    "Laeufer auf Feld (i,j)".
    Constraint: Fuer jede Diagonale hoechstens ein Laeufer
```

```python
    (paarweise Klauseln, AMO-Enkodierung).
    """
    # Schritt 1: Alle Diagonalen sammeln.
    # Schluessel: (typ, index) -> Liste der Felder auf dieser Diagonale.
    diagonals = {}
    for i in range(1, n+1):
        for j in range(1, n+1):
            # Hauptdiagonale: i - j ist konstant.
            diagonals.setdefault(('haupt', i-j), []).append((i, j))
            # Gegendiagonale: i + j ist konstant.
            diagonals.setdefault(('gegen', i+j), []).append((i, j))

    # Schritt 2: Paarweise AMO-Klauseln erzeugen.
    clauses = []
    for feld_liste in diagonals.values():
        if len(feld_liste) < 2:
            continue  # einelementige Diagonalen brauchen keinen Constraint
        for p in range(len(feld_liste)):
            for q in range(p+1, len(feld_liste)):
                (i1, j1) = feld_liste[p]
                (i2, j2) = feld_liste[q]
                klausel = f"(NOT b_{{{i1},{j1}}} OR NOT b_{{{i2},{j2}}})"
                clauses.append(klausel)
    return clauses


def formel_ausgabe(clauses):
    """Verbindet alle Klauseln mit AND zur Gesamtformel."""
    return " AND ".join(clauses)


if __name__ == "__main__":
    n = 4
    klauseln = bishop_independence(n)
    print(f"Anzahl Klauseln fuer n={n}: {len(klauseln)}")
    for k in klauseln[:5]:
        print(k)
    print("...")
```

Schritt 5: Das Skript Zeile für Zeile

Das Dictionary `diagonals`.

- Wir benutzen ein Python-Dictionary (ein „Nachschlagewerk“), in dem der Schlüssel eine Diagonale identifiziert und der Wert eine Liste der Felder dieser Diagonale ist.
- Der Schlüssel ist ein Tupel (typ, index), z. B. `('haupt', 0)` für die Hauptdiagonale mit $i - j = 0$.
- `setdefault(key, [])` liefert die Liste zum Schlüssel zurück – und erzeugt sie, falls

der Schlüssel noch nicht existiert, mit dem Vorgabewert `[]` (leere Liste). Danach hängen wir mit `.append((i,j))` das aktuelle Feld an.

Analogie

`diagonals.setdefault(key, []).append(...)` ist eine klassische Python-Technik zum **Gruppieren**. Stell dir vor, du würfst Briefumschläge in Schubladen mit Etikett. Für jeden Brief (Feld) schaut man, ob die Schublade mit dem passenden Etikett (`('haupt', 0)`) schon existiert. Wenn ja, rein damit. Wenn nein, neue Schublade anlegen und reinlegen.

Die doppelte Schleife über i und j. Wir gehen jedes Feld (i, j) genau einmal durch. Pro Feld tragen wir es in zwei Schubladen ein: einmal in die passende Hauptdiagonale, einmal in die Gegendiagonale. Am Ende der Schleife hat jede Diagonale ihre Liste von Feldern.

Die Klauselerzeugung. Für jede Schublade (also jede Diagonale):

- Wenn die Schublade nur ein Feld enthält (z. B. die Ecke $(1, 4)$ ist auf der Hauptdiagonale $i - j = -3$ allein), gibt es nichts auszuschließen. Wir überspringen sie mit `continue`.
- Sonst generieren wir alle Paare (p, q) mit $p < q$ (das ist eine Standardkonstruktion „paarweise ohne Doppel“). Jedes Paar gibt eine Klausel der Form `(NOT b_{i1,j1} OR NOT b_{i2,j2})`.

Warum `NOT ... OR NOT ...`? Die Übersetzung: $\neg(b_{i_1,j_1} \wedge b_{i_2,j_2})$ ist nach De Morgan dasselbe wie $\neg b_{i_1,j_1} \vee \neg b_{i_2,j_2}$. Beide Schreibweisen sind äquivalent, aber die zweite liegt bereits in CNF vor.

Schritt 6: Konkretes Beispiel $n = 4$

Für $n = 4$ gibt es jeweils $2n-1 = 7$ Hauptdiagonalen und 7 Gegendiagonalen. Ihre Größen sind $1, 2, 3, 4, 3, 2, 1$ in beide Richtungen (die mittelste Diagonale hat $n = 4$ Felder, dann wird sie zu den Ecken hin kleiner).

Anzahl Klauseln pro Diagonale: $\binom{k}{2}$ für Größe k.

- Größe 1: $\binom{1}{2} = 0$ Klauseln (wird übersprungen).
- Größe 2: $\binom{2}{2} = 1$ Klausel.
- Größe 3: $\binom{3}{2} = 3$ Klauseln.
- Größe 4: $\binom{4}{2} = 6$ Klauseln.

Summe pro Richtung: $0 + 1 + 3 + 6 + 3 + 1 + 0 = 14$. Beide Richtungen zusammen: $14 + 14 = \mathbf{28}$ Klauseln.

Beispiel-Klausel. Für die Hauptdiagonale $i-j = 0$, zu der die Felder $(1, 1), (2, 2), (3, 3), (4, 4)$ gehören, erzeugt das Skript die 6 Klauseln

$$\begin{aligned}&\neg(b_{1,1} \wedge b_{2,2}) \;\wedge\; \neg(b_{1,1} \wedge b_{3,3}) \;\wedge\; \neg(b_{1,1} \wedge b_{4,4})\\ &\wedge\; \neg(b_{2,2} \wedge b_{3,3}) \;\wedge\; \neg(b_{2,2} \wedge b_{4,4}) \;\wedge\; \neg(b_{3,3} \wedge b_{4,4}).\end{aligned}$$

Lesart: „Es stehen nicht gleichzeitig Läufer auf $(1, 1)$ und $(2, 2)$“, „nicht gleichzeitig auf $(1, 1)$ und $(3, 3)$“ usw.

Schritt 7: Was heißt „Modell der Formel“?

Ein Modell ist eine Belegung aller $b_{i,j}$ mit 0 oder 1. Eine 1 steht für „hier steht ein Läufer“, eine 0 für „hier steht kein Läufer“.

Triviales Modell: alle $b_{i,j} = 0$. Dann ist jede Klausel der Form $\neg(0 \wedge 0) = \neg 0 = 1$, also alle Klauseln wahr – das leere Brett ist erlaubt.

Nicht-triviales Modell für $n = 4$: Platziere Läufer auf $(1,1), (1,3), (4,2), (4,4)$. Prüfe:

- $(1,1)$: Hauptdiagonale $i - j = 0$, Gegendiagonale $i + j = 2$.
- $(1,3)$: Hauptdiagonale $i - j = -2$, Gegendiagonale $i + j = 4$.
- $(4,2)$: Hauptdiagonale $i - j = 2$, Gegendiagonale $i + j = 6$.
- $(4,4)$: Hauptdiagonale $i - j = 0$ – **Kollision mit $(1,1)$!**

Diese Konfiguration ist also *kein* Modell, weil $(1,1)$ und $(4,4)$ auf derselben Hauptdiagonale liegen. Die Formel verbietet das durch $\neg(b_{1,1} \wedge b_{4,4})$.

Eine gültige Konfiguration: Läufer auf $(1,1), (1,4), (4,1), (4,4)$.

- Hauptdiagonalen: $1 - 1 = 0, 1 - 4 = -3, 4 - 1 = 3, 4 - 4 = 0$. **Kollision: $(1,1)$ und $(4,4)$ auf 0.**

Auch nicht erlaubt. Läufer auf den Ecken teilen sich nämlich die beiden Hauptdiagonalen und Gegendiagonalen paarweise. Es ist gar nicht so leicht, viele Läufer gleichzeitig unterzubringen!

Maximale Läufer-Anzahl

Auf einem $n \times n$-Brett kann man maximal $2n - 2$ unabhängige Läufer platzieren (für $n \geq 2$). Unsere Formel hat aber auch Modelle mit weniger Läufern, bis hin zum leeren Brett. Sie beschreibt nur die *Verträglichkeit*, nicht die Maximalität.

Pattern zum Mitnehmen

Das Python-Idiom

```
dict.setdefault(key, []).append(value)
```

ist Standard zum **Gruppieren von Objekten nach Schlüssel**. Es ersetzt umständliches `if key not in dict: dict[key] = []; dict[key].append(value)` durch einen einzigen Einzeiler.

Zusammenfassung und goldene Regeln

Kurzübersicht der Ergebnisse

Aufgabe	Inhalt	Wichtigstes Ergebnis
1	Sudoku als Formel	64 Variablen, 400 Klauseln, 5 Constraint-Arten
2	Schaltkreis als Formel	Hilfsvariablen o_1, o_2, o_3; 6 Äquivalenzen
2a	$F = 1$ möglich?	Ja, z. B. $(A, B, C) = (1, 1, 0)$
2b	$E = A$ möglich?	Ja, z. B. $(A, B, C) = (1, 1, 0)$ oder $(0, 0, 0)$
2c	$F = 1 \Leftrightarrow D = 0$?	Nein, Gegenbeispiel $(1, 1, 1)$
3	Bishop Independence	Skript erzeugt AMO-Klauseln pro Diagonale
3 ($n = 4$)	Anzahl Klauseln	28 (je 14 Haupt-/Gegendiagonalen)

Goldene Regeln

Goldene Regeln zum Mitnehmen

1. **Encoding-Rezept.** Eine Bool'sche Variable pro Entscheidung. Constraints als Konjunktion von **AMO-**, **ALO-** und **EO**-Teilformeln. Gesamtformel = Konjunktion aller Regeln.

2. **AMO paarweise.** Für „höchstens eine von k Variablen wahr" schreibt man $\bigwedge_{i<j} \neg(x_i \wedge x_j)$. Anzahl: $\binom{k}{2}$ Klauseln. Funktioniert immer, wenn k nicht zu groß wird.

3. **EO = ALO $\wedge$ AMO.** „Genau eine" zerfällt in „mindestens eine" (eine große Disjunktion) und „höchstens eine" (viele Paar-Ausschlüsse).

4. **Schaltkreise.** Pro Gate-Output eine Äquivalenz, alle mit $\wedge$ verbunden. Für interne Drähte Hilfsvariablen einführen. Summen-Bedingungen wie $[> k], [= k]$ mit Fallunterscheidung in Bool'sche Ausdrücke übersetzen.

5. **Fragen an den Erfüllbarkeitstester.**
   - „Kann X passieren?" $\Rightarrow$ SAT-Test auf $\Phi \wedge X$.
   - „Gilt immer Y?" $\Rightarrow$ UNSAT-Test auf $\Phi \wedge \neg Y$.

6. **Schach-Diagonalen.** Hauptdiagonale: $i - j$ konstant (links oben $\rightarrow$ rechts unten). Gegendiagonale: $i + j$ konstant (rechts oben $\rightarrow$ links unten). Je $2n - 1$ Stück auf einem $n \times n$-Brett.

7. **Sanity-Check.** Anzahl der Klauseln und Variablen mitzählen. Falls die Zahlen nicht passen, irgendwo fehlt eine Bedingung oder es gibt Doppelzählungen.

Leitmotiv

Aussagenlogik ist eine **universelle Puzzle-Sprache**. Jedes Problem aus Kombinatorik, Schach oder Schaltkreisbau lässt sich in diese Sprache übersetzen. Einmal übersetzt, steht ein ganzer Baukasten an Werkzeugen bereit: SAT-Solver, Äquivalenzumformungen, Normalformen. Das ist der Grund, warum Logik so viel mehr ist als nur eine mathematische Spielerei – sie ist die Grundlage, auf der Computer Probleme „verstehen“ und lösen können.

Damit sind alle drei Aufgaben des Blattes durchgearbeitet. Wer die drei Schritte *Variablen einführen*, *Regeln als Teilformeln*, *alles mit* $\wedge$ *verbinden* verinnerlicht hat, kann damit weit über Sudoku, Schach und Schaltkreise hinaus gehen. Viel Spaß beim Weiter-Encodieren!